RAG Learning

2.3 Producing Robust Programs

OCR GCSE Computer Science (J277) — Paper 2

> A **robust program** handles unexpected inputs and situations gracefully without crashing, is thoroughly tested to ensure it works correctly, and is written in a way that other programmers can easily read, understand, and maintain. Robustness is not optional — it is a fundamental quality requirement of all professional software.

2.3.1 Defensive Design

Defensive design means writing code that **anticipates problems before they occur**. The goal is to ensure the program behaves correctly even when users make mistakes, enter unexpected data, or deliberately try to break it. A defensive programmer assumes that anything that can go wrong will go wrong.

Anticipating Misuse

Users will inevitably enter unexpected input. A robust program must handle all of these gracefully:

- **Wrong data type** — entering "hello" when a number is expected
- **Out-of-range values** — entering 999 for age, or -5 for a quantity
- **Empty input** — pressing Enter without typing anything
- **Special characters** — entering symbols, SQL code, or HTML tags in text fields
- **Extremely long input** — pasting thousands of characters into a short field
- **Deliberate attacks** — SQL injection, cross-site scripting (XSS), buffer overflow attempts

For each possible misuse, the program should display a **clear, helpful error message** and allow the user to try again — not crash, freeze, or produce wrong results silently.

© RAG Learning · Not for redistribution

Input Validation

Input validation checks that data meets **all requirements** before the program processes it. There are several types of validation check, and a single input may need multiple checks:

Check Type	What It Checks	Example
Range check	Value falls within an acceptable minimum-maximum range	Age must be between 0 and 120. Temperature must be between -50 and 60. Quantity must be 1 or more.
Type check	Data is the correct data type	Age must be an integer (not "twelve" or "17.5"). Price must be a number. Name must not contain digits.
Length check	Input has an acceptable number of characters	Password must be 8-64 characters. Postcode must be 6-8 characters. Phone number must be 11 digits.
Presence check	Field is not empty — something was actually entered	Email field must not be blank. Name field is required. All mandatory fields must be filled.
Format check	Input matches a required pattern or structure	Email must contain '@' and a domain (e.g., user@example.com). Date must be DD/MM/YYYY. Postcode must match UK format (e.g., SW1A 1AA).
Look-up check	Value exists in a predefined list of acceptable values	Country must be from a list of valid countries. Product code must exist in the database. Title must be Mr, Mrs, Ms, Dr, etc.

```
// ── Input validation example (OCR pseudocode) ──────────────
// Combining range check + type check using a DO-UNTIL loop:

do
    age = input("Enter your age (0-120): ")
    if NOT isInteger(age) then
        print("Error: please enter a whole number.")
    elseif int(age) < 0 OR int(age) > 120 then
        print("Error: age must be between 0 and 120.")
```

```
    endif
until isInteger(age) AND int(age) >= 0 AND int(age) <= 120
age = int(age)

// ── Python equivalent with full error handling ──────────────
while True:
    user_input = input("Enter your age (0-120): ")

    # Presence check
    if user_input == "":
        print("Error: this field cannot be empty.")
        continue

    # Type check
    try:
        age = int(user_input)
    except ValueError:
        print("Error: please enter a whole number.")
        continue

    # Range check
    if age < 0 or age > 120:
        print("Error: age must be between 0 and 120.")
        continue

    break  # All checks passed — exit loop

print(f"Your age is {age}")
```

```
// ── Email validation example ──────────────────────────────
function isValidEmail(email)
    // Presence check
    if len(email) == 0 then
        return false
    endif
    // Format check: must contain exactly one '@'
    if email.count("@") != 1 then
        return false
    endif
    // Format check: must have text before and after '@'
    parts = email.split("@")
    if len(parts[0]) == 0 OR len(parts[1]) == 0 then
        return false
```

```
    endif
    // Format check: domain must contain a '.'
    if "." not in parts[1] then
        return false
    endif
    return true
endfunction

# Python:
def is_valid_email(email):
    if len(email) == 0:
        return False
    if email.count("@") != 1:
        return False
    local, domain = email.split("@")
    if len(local) == 0 or len(domain) == 0:
        return False
    if "." not in domain:
        return False
    return True
```

Authentication

Authentication is the process of verifying a user's identity before granting access to the system or specific features. It answers the question: "Are you who you claim to be?"

- **Username and password** — the most common method. The system compares the entered credentials against stored records. Passwords should be stored as **hashes** (one-way mathematical transformations), not as plain text. If the database is breached, hashed passwords cannot be easily reversed to reveal the original password.
- **Two-factor authentication (2FA)** — requires two different types of evidence: something you **know** (password) + something you **have** (phone/security key) or something you **are** (fingerprint/face). Much more secure because an attacker needs both factors.
- **Biometric authentication** — fingerprint scanner, facial recognition, retina scan, voice recognition. Unique to each person and cannot be forgotten or shared. However, biometric data cannot be changed if compromised (unlike a password).
- **CAPTCHA** — "Completely Automated Public Turing test to tell Computers and Humans Apart." Prevents automated bots from creating accounts, submitting forms, or brute-forcing passwords.

Presents a challenge (distorted text, image selection, puzzle) that is easy for humans but difficult for software.

```
// Simple authentication pseudocode:
const MAX_ATTEMPTS = 3
attempts = 0
authenticated = false

while attempts < MAX_ATTEMPTS AND authenticated == false
    username = input("Username: ")
    password = input("Password: ")
    if checkCredentials(username, password) == true then
        authenticated = true
        print("Login successful. Welcome, " + username)
    else
        attempts = attempts + 1
        remaining = MAX_ATTEMPTS - attempts
        print("Invalid credentials. " + str(remaining) + " attempts remaining.")
    endif
endwhile

if authenticated == false then
    print("Account locked. Contact administrator.")
endif

# Python:
MAX_ATTEMPTS = 3
for attempt in range(MAX_ATTEMPTS):
    username = input("Username: ")
    password = input("Password: ")
    if check_credentials(username, password):
        print(f"Login successful. Welcome, {username}")
        break
    remaining = MAX_ATTEMPTS - attempt - 1
    print(f"Invalid credentials. {remaining} attempts remaining.")
else:
    print("Account locked. Contact administrator.")
```

Exam Tip: Validation checks data is **reasonable and in the correct format** (is this a valid email address?). Authentication checks the **user's identity** (are you who you claim to be?). These are different concepts — do not confuse them. Validation prevents bad data; authentication

prevents unauthorised access. For validation questions, always name the **specific type of check** (range check, type check, etc.) and explain what it checks.

2.3.2 Maintainability

Code that is easy to **read, understand, and modify** by other programmers (or by yourself months later) is said to be maintainable. In professional software development, code is read far more often than it is written. Multiple programmers work on shared codebases over years. Unmaintainable code becomes a liability — changes take longer, bugs are harder to find, and the cost of development escalates.

Technique	Why It Matters	Good vs Bad Example
Meaningful names	Names should describe the purpose of the variable, function, or constant. Self-documenting code is code that explains itself through clear naming. Avoids needing excessive comments.	Good: `totalScore`, `studentName`, `calculateTax(income)`, `isGameOver` Bad: `ts`, `sn`, `ct(i)`, `flag`, `x`, `temp2`
Comments	Explain **what a block does and WHY**, not obvious single lines. Future programmers need context for complex logic. Comment the intent, not the mechanics.	Good: `// Apply 20% discount for loyalty members` Bad: `// add 1 to x` (this is obvious from the code itself and adds no value)
Indentation	Visually shows the **structure** of the code. Makes it immediately clear which lines are inside loops, conditions, and functions. Logic errors (like code accidentally outside a loop) become visible.	All code inside an IF block is indented one level; nested loops are indented further. Consistent indentation (always 4 spaces or always 1 tab, never mixed).
Subroutines	Break code into small, focused, reusable modules. Each subroutine does **one thing well**. The main	Good: `readData()`, `processScores()`, `displayResults()`

	program reads as a high-level overview.	Bad: One giant main function with 500 lines doing everything
Constants	Give meaningful names to fixed values. `const MAX_LIVES = 3` is clearer than magic number `3` scattered through code. Change value in one place, not dozens.	Good: `const VAT_RATE = 0.2` then `tax = price * VAT_RATE` Bad: `tax = price * 0.2` (what is 0.2? Why 0.2?)
Consistent style	Follow a consistent naming convention (camelCase or snake_case), consistent spacing, consistent brace placement. Mixed styles make code harder to scan.	Pick one style and stick to it throughout the entire project.

```
// ── POORLY MAINTAINED CODE ──────────────────────────────
x=0
for i=0 to 9
y=int(input(""))
if y>x then
x=y
endif
next i
print(x)
// What does this do? Hard to tell without careful analysis.

// ── WELL MAINTAINED CODE (same logic) ────────────────────
// Find and display the highest score from 10 user inputs
const NUM_ENTRIES = 10
highestScore = 0

for i = 0 to NUM_ENTRIES - 1
    score = int(input("Enter score " + str(i + 1) + ": "))
    if score > highestScore then
        highestScore = score
    endif
next i

print("The highest score is: " + str(highestScore))
// Clear names, comments, constants, indentation — easy to understand.
```

2.3.3 Testing — Types of Test Data

Every program must be tested with carefully chosen test data to verify it works correctly under all conditions. There are three essential categories of test data. A thorough test plan includes examples of all three.

Type	Purpose	Example (valid range: 1-100)
Normal (valid) data	Confirm the program works correctly with **typical, expected input** that falls comfortably within the valid range. Tests the "happy path" — what happens when everything is correct.	Enter **50** — should be accepted and processed normally. Also test: 25, 75, 42 (various typical values).
Boundary (edge) data	Test values at the **very edges** of the acceptable range — both just inside and just outside. Off-by-one errors (using > instead of >=, or < instead of <=) are the most common logic bugs, and boundary testing catches them.	Enter **1** (lowest valid) — should be accepted. Enter **100** (highest valid) — should be accepted. Enter **0** (just below range) — should be rejected. Enter **101** (just above range) — should be rejected.
Erroneous (invalid) data	Confirm the program **rejects bad input gracefully** with a clear error message, without crashing, freezing, or producing wrong results silently.	Enter **-5** (negative number). Enter **999** (way out of range). Enter **"hello"** (wrong data type). Enter **""** (empty/blank). Enter **3.5** (decimal when integer expected).

© RAG Learning · Not for redistribution

		All should display appropriate error messages.

Writing a Complete Test Plan

A test plan documents what will be tested, with what data, and what the expected result should be. During testing, the actual result is compared to the expected result. Any mismatch indicates a bug.

```
// Test plan for: "Enter a percentage mark (0-100), display grade"
// A: 70-100, B: 60-69, C: 50-59, Fail: 0-49

| Test | Input   | Type      | Expected Result                    | Pass? |
|------|--------|-----------|------------------------------------|-------|
| 1    | 75      | Normal    | Accepted, displays "Grade: A"      |       |
| 2    | 55      | Normal    | Accepted, displays "Grade: C"      |       |
| 3    | 30      | Normal    | Accepted, displays "Grade: Fail"   |       |
| 4    | 0       | Boundary  | Accepted, displays "Grade: Fail"   |       |
| 5    | 100     | Boundary  | Accepted, displays "Grade: A"      |       |
| 6    | 50      | Boundary  | Accepted, displays "Grade: C"      |       |
| 7    | 49      | Boundary  | Accepted, displays "Grade: Fail"   |       |
| 8    | 70      | Boundary  | Accepted, displays "Grade: A"      |       |
| 9    | 69      | Boundary  | Accepted, displays "Grade: B"      |       |
| 10   | -1      | Boundary  | Rejected with error message        |       |
| 11   | 101     | Boundary  | Rejected with error message        |       |
| 12   | -50     | Erroneous | Rejected with error message        |       |
| 13   | 200     | Erroneous | Rejected with error message        |       |
| 14   | "abc"   | Erroneous | Rejected with error message        |       |
| 15   | ""      | Erroneous | Rejected with error message        |       |
| 16   | 72.5    | Erroneous | Rejected (not whole number)        |       |

Note: boundary tests check EVERY boundary between grades (49/50,
69/70) plus the overall range boundaries (0, 100, -1, 101).
```

Exam Tip: Always test **BOTH sides of every boundary**. If the valid range is 1-10, test: 0 (invalid), 1 (valid boundary), 10 (valid boundary), and 11 (invalid). If there are internal boundaries (e.g., grade boundaries at 50, 60, 70), test both sides of each: 49 and 50, 59 and 60,

> 69 and 70. This is where off-by-one bugs hide. In exam test plans, include at least one normal, two boundary pairs, and two erroneous values.

2.3.4 Testing Strategies

Strategy	When	Description	Advantage
Iterative testing	Throughout development (ongoing, continuous)	Each module or subroutine is tested **as it is written**, before moving on to the next. Errors are caught immediately while the code is fresh in the programmer's mind. Individual components are tested in isolation.	Bugs found early are cheaper and easier to fix. The programmer knows exactly which code they just changed, so the source of a bug is easy to identify. Prevents errors from accumulating.
Final (terminal) testing	After development is complete	Testing of the **complete, integrated program**. Checks that all parts work together correctly (integration). Tests every execution path. Includes User Acceptance Testing (UAT) where real end-users test the system against their requirements.	Verifies the whole system works as intended. Catches integration bugs (where individual modules work alone but fail together). UAT ensures the system meets user expectations.

Both strategies are needed. Iterative testing alone misses integration issues. Final testing alone catches bugs too late when they are expensive to fix. Professional development uses both: iterative testing during development (often automated as unit tests), followed by final integration and acceptance testing.

Debugging Tools and Techniques

- **Print statements:** Insert temporary `print()` statements to display variable values at key points. Simple but effective for small programs. Remove or comment out after debugging.

- **Trace tables:** Manually track variable values step by step through the algorithm on paper. Excellent for finding logic errors in loops and conditions.
- **Debugger (IDE feature):** Step through code one line at a time, inspecting variable values at each step. Set breakpoints to pause execution at specific lines. Far more powerful than print statements for complex programs.
- **Watch windows:** Monitor specific variable values continuously as the program executes in the debugger.
- **Test logs:** Record all test results systematically to track which tests pass and which fail, making it easy to verify fixes.

2.3.4b White-Box and Black-Box Testing

Testing approaches can be classified by whether the tester can see the internal code of the program. **White-box testing** (also called structural or glass-box testing) tests the program with full knowledge of its internal structure. **Black-box testing** (also called functional or behavioural testing) tests the program based purely on its specification — the tester has no knowledge of how the code works internally, only what inputs and outputs are expected.

Feature	White-Box Testing	Black-Box Testing
Access to code?	**Yes** — tester can see, read, and analyse the source code	**No** — tester only knows the specification (what the program should do), not how it does it
Who performs it?	Usually the **developer** who wrote the code, or another developer on the same team. Requires programming knowledge.	Usually a **separate test team**, the client, or end-users. Does not require programming knowledge — only knowledge of the requirements.
What it tests	Internal logic: every branch of every IF statement is executed; every loop runs at least once; all code paths are covered.	Inputs and outputs: does the program meet its specification? Given this input, does it produce the correct output?

© RAG Learning · Not for redistribution

	Checks that the *implementation* is correct.	Checks that the *behaviour* is correct from the user's perspective.
Test design basis	Based on the **structure of the code** — test cases are designed to exercise specific lines, branches, or paths through the code.	Based on the **specification / requirements** — test cases are designed from the expected behaviour without looking at the code.
What it finds	Logic errors in specific code paths; dead code (code that can never be reached); untested branches; security vulnerabilities in the implementation.	Missing functionality; incorrect outputs; failures to meet the specification; issues the user would actually notice.
Limitations	The developer may have the same blind spots as when writing the code — if a feature was forgotten entirely, it will not be tested. Difficult for complex programs (many possible paths).	Cannot test internal logic — a function could work correctly from the outside even while containing internal errors that will cause problems in edge cases not covered by the spec.
Examples	Checking that every branch of an IF-ELSEIF-ELSE chain is executed at least once; verifying a loop terminates correctly; checking that an error-handling path is actually reachable.	Entering test data and checking the output matches the expected result from the specification; UAT (User Acceptance Testing) where real users test the system against their requirements.

```
Example: Testing a function that calculates a student's grade (0-100 → A/B/C/Fail)

WHITE-BOX test design (based on the code):
  The tester reads the code and sees:
    if score >= 70: grade = "A"
    elif score >= 60: grade = "B"
    elif score >= 50: grade = "C"
    else: grade = "Fail"
  Test cases designed to execute EVERY branch:
    Test 1: score = 75 → should enter first branch (A)
    Test 2: score = 65 → should enter second branch (B)
    Test 3: score = 55 → should enter third branch (C)
    Test 4: score = 40 → should enter else branch (Fail)
```

© RAG Learning · Not for redistribution

```
    Test 5: score = 70 → boundary of first branch (A)
    Test 6: score = 69 → boundary between A and B (B)
  Every line and every branch is covered.

BLACK-BOX test design (based on the specification):
  The tester only knows the spec: "accepts 0-100, outputs A (70-100),
  B (60-69), C (50-59), Fail (0-49), rejects out-of-range values"
  Test cases based on the SPEC, not the code:
    Test 1: 75 → expect "A"          (normal, A range)
    Test 2: 55 → expect "C"          (normal, C range)
    Test 3: 100 → expect "A"         (boundary, top)
    Test 4: 0   → expect "Fail"      (boundary, bottom)
    Test 5: 101 → expect error       (erroneous)
    Test 6: -1  → expect error       (erroneous)
    Test 7: "abc" → expect error     (erroneous, wrong type)
    Test 8: 70  → expect "A"         (boundary between A and B)
    Test 9: 69  → expect "B"         (boundary between A and B)
  The tester never looks at the code — only verifies the outputs
  match the specification.
```

Exam Tip: White-box = tester **can see the code** (tests internal logic, code paths, branches). Black-box = tester **cannot see the code** (tests inputs and outputs against the specification). In practice, both are used: developers use white-box testing during development (iterative testing) to ensure all code paths work; independent testers and users use black-box testing to verify the program meets its requirements. A key exam point: black-box testing is better for *finding missing features* because the tester is focused on the spec; white-box testing is better for *finding unreachable code* and *logic errors in specific branches*.

2.3.5 Types of Errors

Programs can contain three types of errors. Understanding the differences is essential for debugging:

Error Type	What It Is	When Detected	Examples	How to Fix
Syntax error	The code breaks the grammatical rules of	At compile/interpretation	Missing colon after `if` in	Read the error

	the programming language. The program **cannot run at all** — the translator (compiler/interpreter) refuses to process it.	time (before the program runs). The translator highlights the error with a line number and description.	Python; misspelled keyword (`prnit` instead of `print`); mismatched brackets or quotes; missing `endif` or `endwhile` ; using `=` instead of `==` for comparison (in some languages).	message carefully — it usually tells you the line number and what is wrong. Fix the syntax and try again.
Logic error	The code runs without crashing but produces **wrong results**. The algorithm or formula is incorrect. The hardest type to find because there is no error message.	During testing, by comparing actual output to expected output. The program appears to work but gives incorrect answers.	Using `+` instead of `-` ; wrong comparison (`>` instead of `>=`); loop running one too many or too few times (off-by-one); using the wrong variable; conditions in the wrong order; calculating average as `total * count` instead of `total / count` .	Use trace tables, print statements, or a debugger to track variable values step by step. Compare actual behaviour to expected behaviour. Check formulas and conditions carefully.
Runtime error	The program starts running but **crashes**	During execution. The program halts with an	Division by zero (`x / 0`);	Add validation

	during execution when it attempts an impossible operation.	error message (exception/traceback).	accessing an array index that does not exist (`list[10]` when list has 5 elements); trying to open a file that does not exist; converting invalid text to a number (`int("hello")`); stack overflow from infinite recursion; running out of memory.	to prevent the impossible operation (check divisor is not zero, check index is in range, check file exists). Use try/except (Python) to catch and handle exceptions gracefully.

```
// ── SYNTAX ERROR ──────────────────────────────────────────
# Python:
if age > 18                    # Missing colon → SyntaxError
    print("Adult")

# Fixed:
if age > 18:                   # Colon added
    print("Adult")

// ── LOGIC ERROR ───────────────────────────────────────────
# Calculating average incorrectly:
total = 10 + 20 + 30           # = 60
count = 3
average = total * count        # = 180 (WRONG! Should be total / count)

# Fixed:
average = total / count        # = 20 (correct)

// ── RUNTIME ERROR ─────────────────────────────────────────
# Division by zero:
marks = 0
```

```
average = total / marks        # ZeroDivisionError! Program crashes.

# Fixed with validation:
if marks != 0:
    average = total / marks
else:
    print("Error: cannot divide by zero")

# Fixed with exception handling:
try:
    average = total / marks
except ZeroDivisionError:
    print("Error: cannot calculate average with zero marks")
```

Exam Tip: The key distinctions: **Syntax error** = program cannot run at all (detected by the translator before execution). **Logic error** = program runs but produces wrong results (no error message — detected only by testing). **Runtime error** = program crashes during execution (error message/traceback at the point of failure). A common exam question gives a piece of code and asks you to identify what type of error it contains and how to fix it. Logic errors are the hardest to find because the program does not crash — it just silently gives wrong answers.

© RAG Learning · Not for redistribution